Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №8
на тему
**«СТЕГАНОГРАФИЧЕСКИЕ МЕТОДЫ.
СОКРЫТИЕ СООБЩЕНИЯ В ЧАСТОТНОЙ ОБЛАСТИ JPEG»**
БГУИР 6-05-0612-02 21

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ПОЛЯКОВ Кирилл Борисович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Краткие теоретические сведения 4
2.1 Стеганография и криптография 4
2.2 JPEG и дискретное косинусное преобразование 4
2.3 Квантование с индексом модуляции 4
2.4 Метрика качества 4
3 Выполнение работы 6
3.1 Структура программного средства 6
3.2 Схема встраивания 6
3.3 Прямое и обратное преобразование 6
3.4 Встраивание 7
3.5 Проверка контейнера 7
3.6 Извлечение 7
3.7 Интерфейс 8
3.8 Тестирование 10
3.9 Ёмкость и скорость 11
Заключение 13
Список использованных источников 14
Приложение А (Обязательное) Листинг программного кода 15

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы является изучение стеганографических методов сокрытия информации в частотной области изображения и получение практических навыков работы с дискретным косинусным преобразованием (ДКП).

В ходе работы требовалось реализовать программное средство, которое:

1 скрывает текстовое сообщение в JPEG-изображении, изменяя коэффициенты ДКП блоков 8×8;

2 извлекает сообщение обратно из сохранённого контейнера;

3 сохраняет контейнер без потерь и проверяет, что после кодирования PNG сообщение читается;

4 показывает в терминальном интерфейсе исходное изображение, контейнер и усиленную разность между ними, чтобы изменение было видно пользователю;

5 определяет ёмкость изображения и отказывается встраивать сообщение, которое в неё не помещается;

6 проверяется модульными тестами, включая круговое встраивание и извлечение.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Стеганография и криптография

Криптография скрывает **содержание** сообщения, но не факт его передачи: шифртекст виден наблюдателю. Стеганография скрывает сам факт наличия сообщения, встраивая его в безобидный контейнер — изображение, звук или видео [1]. Оба подхода дополняют друг друга: полезную нагрузку можно сначала зашифровать, а затем спрятать.

Для изображения важны три противоречащих друг другу свойства: ёмкость (сколько данных можно спрятать), незаметность (насколько искажается картинка) и устойчивость (переживёт ли сообщение сжатие или редактирование). Улучшение одного свойства обычно ухудшает два других.

2.2 JPEG и дискретное косинусное преобразование

JPEG переводит пиксели из RGB в яркость и две цветоразностные компоненты, делит яркость на блоки 8×8 и применяет к каждому блоку ДКП [2]:

$$F(u,v) = \frac{1}{4}C(u)C(v)\sum_{x=0}^{7}\sum_{y=0}^{7} f(x,y)\cos\left(\frac{(2x+1)u\pi}{16}\right)\cos\left(\frac{(2y+1)v\pi}{16}\right), \quad (2.1)$$

где $C(0) = 1/\sqrt{2}$, а $C(u>0) = 1$. Обратное преобразование имеет тот же вид и возвращает блок из спектра. Низкие частоты отвечают за плавные изменения яркости, высокие — за мелкие детали. Коэффициенты затем квантуются: менее важные частоты округляются грубее.

Именно квантованные коэффициенты и являются естественным местом сокрытия: небольшое изменение среднечастотного коэффициента почти не влияет на вид блока, но переживает некоторые преобразования лучше, чем младшие биты пикселей.

2.3 Квантование с индексом модуляции

В методе QIM значение коэффициента c квантуется с шагом Q: $k = \mathrm{round}\left(\frac{c}{Q}\right)$. Номер ячейки k заменяется на ближайший номер k' с нужной чётностью, а коэффициент переносится в центр этой ячейки: $c' = k'Q$ [3]. При извлечении достаточно повторить квантование и прочитать чётность. Расстояние до границы решения равно $Q/2$, поэтому небольшая ошибка округления не переворачивает бит.

2.4 Метрика качества

Среднеквадратическая ошибка и пиковое отношение сигнал/шум считаются по всем каналам:

$$\mathrm{MSE} = \frac{1}{3WH} \sum_{\text{пикс}} (R - R')^2 + (G - G')^2 + (B - B')^2, \qquad (2.2)$$

$$\mathrm{PSNR} = 10 \lg\left(255^2 / \mathrm{MSE}\right). \qquad (2.3)$$

Чем выше PSNR, тем меньше заметно искажение. Для стеганографии обычно приемлемо значение выше 40 дБ; разница в 1–2 уровня яркости на глаз не видна.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Структура программного средства

Алгоритм вынесен в пакет *internal/crypto/stego* и не зависит от интерфейса:

– *dct.go* — прямое и обратное ДКП блока 8×8;

– *stego.go* — встраивание, извлечение, кадрирование, статистика и проверка контейнера;

– *internal/ui/stegopane* — вкладка *Stego* терминального интерфейса;

– *main.go* — регистрация вкладки рядом с остальными алгоритмами.

Встраивание и извлечение работают с яркостью, вычисляемой одинаково с обеих сторон:

$$Y = 0.299R + 0.587G + 0.114B. \tag{3.1}$$

Контейнер записывается в PNG. Для каждого пикселя к R, G и B прибавляется одна и та же разность $\Delta = Y' - Y$, поэтому оттенок сохраняется, а яркость результата совпадает с восстановленной яркостью блока с точностью до округления.

3.2 Схема встраивания

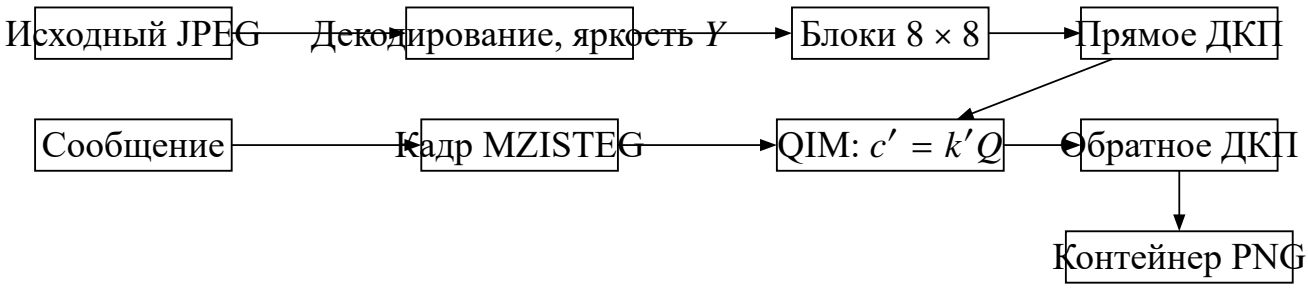


Рисунок 3.1 – Схема встраивания сообщения в коэффициенты ДКП

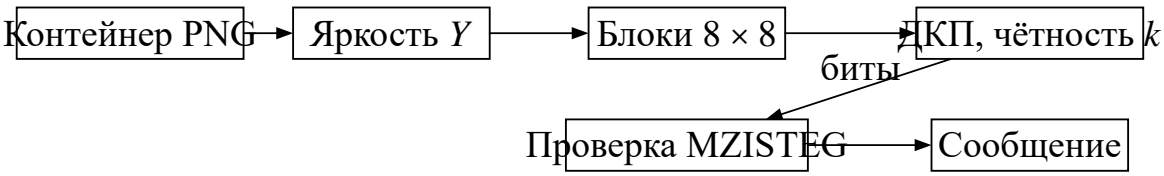


Рисунок 3.2 – Схема извлечения сообщения

3.3 Прямое и обратное преобразование

Функции *forwardDCT* и *inverseDCT* реализуют формулы ДКП раздельно: сначала суммирование по строкам, затем по столбцам. Таблица косинусов 8 × 8

вычисляется один раз при инициализации пакета. Тест *TestDCTRoundTrip* восстанавливает случайный блок и требует максимальную ошибку меньше 10^{-9}.

3.4 Встраивание

Кадр сообщения состоит из семи байт сигнатуры `MZISTEG`, байта версии, четырёх байт длины в порядке big-endian и самой полезной нагрузки. Биты берутся старшим вперёд. Блоки обходятся построчно, внутри блока используются два среднечастотных коэффициента: $(3, 4)$ и $(4, 3)$.

Каждый бит определяет чётность номера квантованной ячейки. Если текущая чётность не совпадает с битом, выбирается соседний номер — тот, что ближе к исходному значению. Затем коэффициент переносится в центр выбранной ячейки. Шаг $Q = 16$ выбран как компромисс: изменение коэффициента не превышает 8, а расстояние до границы решения достаточно велико, чтобы округление при обратном ДКП и сохранении PNG не перевернуло бит.

Ёмкость изображения равна $2 \cdot \lfloor W/8 \rfloor \cdot \lfloor H/8 \rfloor$ бит: по два бита на каждый полный блок. Неполные блоки по правому и нижнему краям не используются. Для изображения 512×384 это 6144 бита, то есть 768 байт вместе с заголовком. Если кадр не помещается, возвращается ошибка *ErrPayloadTooLarge*, а интерфейс показывает занятость ёмкости в процентах.

3.5 Проверка контейнера

Сразу после встраивания *Embed* кодирует результат в PNG в памяти, декодирует обратно, извлекает сообщение и сравнивает с исходным. При расхождении возвращается ошибка *ErrSelfCheck* и файл не создаётся. Это важно для изображений со светлыми участками: после ограничения каналов значением 255 фактическая яркость может отличаться от расчётной. Проверка превращает редкий сбой в понятную ошибку вместо повреждённого контейнера.

Повторное сохранение контейнера в JPEG не поддерживается: квантование JPEG заново округляет коэффициенты. В эксперименте сообщение пережило качество 80, но разрушилось при 70 и ниже; полагаться на это нельзя, поэтому программа всегда пишет PNG.

3.6 Извлечение

Extract повторяет разбиение на блоки и ДКП, читает чётность двух коэффициентов каждого блока и собирает биты в байты. Затем проверяются сигнатура, версия и длина. Обычная фотография не содержит сигнатуры, поэтому для неё возвращается ошибка *ErrNoPayload*, а не случайный набор байтов. Слишком большая заявленная длина также отвергается.

3.7 Интерфейс

Вкладка *Stego* повторяет визуальный язык остальных вкладок и состоит из четырёх полос. Сверху выбирается файл (`.jpg`, `.jpeg`, `.png`) и показываются ёмкость и занятость. В середине слева находится поле сообщения, справа — статистика: PSNR, число изменённых пикселей, максимальная разность яркости, использованные блоки и путь контейнера. Ниже расположены режим *Embed*/*Extract*, кнопки запуска и сохранения сообщения. Основную часть экрана занимает полоса превью. В режиме *Embed* она делится на три колонки: исходное изображение, контейнер и разность яркостей, усиленная в 20 раз. В режиме *Extract* показывается одно превью *CONTAINER* на всю ширину: сравнивать нечего, а контейнер нужно рассмотреть целиком.

Именно разность яркостей является главным доказательством того, что картинка действительно изменилась: на глаз контейнер неотличим от оригинала, а усиленная разность показывает изменённые блоки. Превью отрисовываются библиотекой *ntcharts/picture*. При запуске панель запрашивает у терминала поддержку графического протокола Kitty; если терминал её подтверждает (Kitty, Ghostty, WezTerm, iTerm2), превью переключаются в режим Kitty и показывают полноценное изображение, иначе остаётся универсальный режим полублоков. При изменении размера терминала панель заново задаёт сетку ячеек через *SetSize*, поэтому изображения не сохраняют старую геометрию. Декодирование файла, встраивание, запись контейнера и построение разности выполняются в отдельной горутине и возвращаются в интерфейс сообщением, поэтому на большой фотографии окно не замирает: статус показывает *Embedding…*, а клавиши продолжают работать.



Рисунок 3.3 – Вкладка *Stego* в режиме встраивания: оригинал, контейнер и разность, усиленная в 20 раз. Снимок сделан в tmux, где графический протокол недоступен, поэтому превью показаны в универсальном режиме полублоков



Рисунок 3.4 – Вкладка *Stego* в режиме извлечения: контейнер и восстановленное сообщение (тот же режим полублоков)

Клавиша *Tab* переключает поля, *Enter* открывает выбор файла или запускает действие, *Alt+D* очищает сообщение, *Ctrl+D* переносит извлечённый текст в поле сообщения. *Esc* закрывает выбор файла, а вне его — завершает программу.

Кнопка *SAVE CONTAINER* открывает диалог имени файла: контейнер записывается туда, куда укажет пользователь, а не только рядом с исходным изображением. Клавиша *f* разворачивает превью на весь экран и переключает его по кругу — оригинал, контейнер, усиленная разность; *Esc* в этом режиме возвращает обычную раскладку. При полноэкранном показе превью получает размер всего окна через *SetSize*, поэтому изображение перерисовывается крупнее, а не растягивается из прежней сетки.

Сообщение можно не только набрать, но и загрузить из текстового файла: сочетание *Ctrl+O* открывает в панели *MESSAGE* выбор файла с расширениями `.txt`, `.md`, `.go` и `.json` выбранное содержимое читается в отдельной горутине и подставляется в поле, а объём сообщения сразу виден в счётчике *PAYLOAD*.

3.8 Тестирование

Модульные тесты пакета *internal/crypto/stego* запускаются командой `go test ./internal/crypto/stego`:

– *TestDCTRoundTrip* — точность обратного преобразования;

– *TestEmbedExtractRoundTrip* — пустое, короткое, двоичное и почти максимальное сообщение;

– *TestContainerPNGRoundTrip* — извлечение после кодирования и декодирования PNG;

– *TestExtractCleanImageRejected* — чистая фотография отвергается с *ErrNoPayload*

– *TestPayloadTooLarge* и *TestCapacityBoundary* — граница ёмкости;

– *TestPSNRForOneKilobytePayload* — 1 КБ на изображении 768×512 даёт 44,19 дБ;

– *TestEmbedIsDeterministic* — одинаковый вход даёт одинаковые пиксели;

– *TestJPEGReEncodingAtLowQualityDestroysPayload* — повторное JPEG-сжатие качества 70 разрушает сообщение.

Зависимость качества от размера сообщения для изображения 512×384:

Таблица 3.1 – Размер сообщения, PSNR и число изменённых пикселей

Сообщение, Б	Кадр, Б	PSNR, дБ	Изменено пикселей	Блоков
0	12	62,00	1772	48
32	44	56,65	6726	176
128	140	51,90	20 752	560
256	268	49,13	39 680	1072
512	524	46,20	76 896	2096
756	768	44,58	112 294	3072

Даже при полной загрузке ёмкости PSNR остаётся выше 44 дБ, а максимальная разность яркости не превышает шести уровней. Это подтверждает, что изменение незаметно, хотя формально меняется больше сотни тысяч пикселей.

3.9 Ёмкость и скорость

Ёмкость растёт пропорционально площади: два бита на каждый полный блок 8×8, то есть $2 \cdot \left\lfloor \frac{W}{8} \right\rfloor \cdot \left\lfloor \frac{H}{8} \right\rfloor$ бит. Отсюда

– 512 × 384 — 3072 блока, 768 байт;

– 2400 × 1800 — 67 500 блоков, 16 875 байт;

– 3840 × 2160 (4K) — 129 600 блоков, 32 400 байт.

Число строк блоков не зависит от числа столбцов, поэтому обработка разбита на полосы по строкам и идёт параллельно на всех ядрах; блоки не пересекаются, синхронизация не нужна. Пиксели читаются и пишутся прямо по слайсу *Pix* изображения *image.RGBA*, без интерфейсных вызовов *At* и *Set* на каждый из восьми миллионов пикселей. Измерения на 16-ядерном процессоре: 4K — 0,47 с на встраивание (было 2,74 с до оптимизации) и 0,26 с на запись PNG, 2400 × 1800 — 0,28 с и 0,18 с. Эти операции выполняются в отдельной горутине, поэтому интерфейс остаётся отзывчивым и на 4K.

Отдельно исправлено кеширование превью: модель *picture* хранит отрисованный кадр внутри себя, а панель передавала её по значению, из-за чего кадр пересчитывался при каждой перерисовке — на 4K это давало секунды работы на одно нажатие клавиши. Теперь модели хранятся по указателю и кадр переиспользуется.

ЗАКЛЮЧЕНИЕ

В ходе работы реализовано стеганографическое средство на основе ДКП JPEG. Сообщение встраивается методом QIM в два среднечастотных коэффициента каждого блока 8×8, кадрируется сигнатурой `MZISTEG` и сохраняется в PNG, поскольку повторное JPEG-сжатие разрушает коэффициенты. Встраивание проверяется круговым путём через PNG в памяти, поэтому повреждённый контейнер не создаётся.

Интерфейс показывает исходное изображение, контейнер и усиленную разность яркостей вместе с PSNR и числом изменённых пикселей. Это делает результат наглядным: контейнер визуально совпадает с оригиналом, а разностное превью показывает, где именно прошло встраивание.

Ограничением является малая ёмкость: два бита на блок 8×8 дают 768 байт для изображения 512×384 (для 4K — 32 400 байт). Для больших сообщений нужно использовать больше коэффициентов или изображение большего размера. Кроме того, сообщение не шифруется, поэтому при необходимости его следует сначала зашифровать, а затем скрыть.

Второе ограничение — области чистого чёрного или белого цвета: там яркость нельзя сдвинуть, не обрезав пиксель по границе диапазона, и бит теряется. Встраивание проверяется круговым путём, поэтому такой контейнер не создаётся; в этом случае программа сообщает, что в изображении есть области без запаса по яркости, и предлагает выбрать другую картинку.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Fridrich, J. Steganography in Digital Media: Principles, Algorithms, and Applications / J. Fridrich. – Cambridge: Cambridge University Press, 2010. – 437 с.

[2] Information technology — Digital compression and coding of continuous-tone still images: Requirements and guidelines. – 1992.

[3] Chen, B. Quantization Index Modulation: A Class of Provably Good Methods for Digital Watermarking and Information Embedding / B. Chen, G.W. Wornell // IEEE Transactions on Information Theory. – 2001. – Т. 47, № 4. – С. 1423-1443.

[4] Rabin, M.O. Digitalized Signatures and Public-Key Functions as Intractable as Factorization / M.O. Rabin. – 1979.

[5] Menezes, A.J. Handbook of Applied Cryptography / A.J. Menezes, P.C. van Oorschot, S.A. Vanstone. – Boca Raton: CRC Press, 1996. – 780 с.

[6] Лабораторные работы по дисциплине «Методы защиты информации» : методические указания. – 2024.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – Прямое и обратное дискретное косинусное преобразование

```
package stego

import "math"

// blockSide — сторона блока JPEG. В стандарте изображение делится на блоки
// 8×8, и дискретное косинусное преобразование применяется к каждому блоку.
const blockSide = 8

// dctBasis[u][x] = C(u) * cos((2x+1)uπ/16), где C(0) = 1/√2, C(u>0) = 1.
// Таблица считается один раз: базисные функции от блока не зависят.
var dctBasis = newDCTBasis()

func newDCTBasis() [blockSide][blockSide]float64 {
  var basis [blockSide][blockSide]float64

  for u := 0; u < blockSide; u++ {
    scale := 1.0
    if u == 0 {
      scale = 1 / math.Sqrt2
    }
    for x := 0; x < blockSide; x++ {
      basis[u][x] = scale * math.Cos(
        float64(2*x+1)*float64(u)*math.Pi/float64(2*blockSide),
      )
    }
  }

  return basis
}

// forwardDCT вычисляет двумерное DCT-II блока 8×8.
//
//  F(u,v) = 1/4 · C(u)C(v) · Σx Σy f(x,y) cos((2x+1)uπ/16) cos((2y+1)vπ/16)
//
// Преобразование разделимо, поэтому сначала считаются суммы по строкам
// (горизонтальная частота u), затем по столбцам (вертикальная частота v).
func forwardDCT(block [blockSide][blockSide]float64) [blockSide]
[blockSide]float64 {
  var rows [blockSide][blockSide]float64

  for y := 0; y < blockSide; y++ {
    for u := 0; u < blockSide; u++ {
      var sum float64
      for x := 0; x < blockSide; x++ {
        sum += block[y][x] * dctBasis[u][x]
      }
      rows[y][u] = sum
    }
  }

  var coefficients [blockSide][blockSide]float64

  for v := 0; v < blockSide; v++ {
    for u := 0; u < blockSide; u++ {
      var sum float64
      for y := 0; y < blockSide; y++ {
```

```
        sum += rows[y][u] * dctBasis[v][y]
      }
      coefficients[u][v] = sum / 4
    }
  }

  return coefficients
}

// inverseDCT восстанавливает блок 8×8 из его спектра:
//
//  f(x,y) = 1/4 · Σu Σv C(u)C(v) F(u,v) cos((2x+1)uп/16) cos((2y+1)vп/16)
//
// Как и прямое преобразование, обратное выполняется разделимо.
func  inverseDCT(coefficients  [blockSide][blockSide]float64)  [blockSide]
[blockSide]float64 {
  var columns [blockSide][blockSide]float64

  for v := 0; v < blockSide; v++ {
    for x := 0; x < blockSide; x++ {
      var sum float64
      for u := 0; u < blockSide; u++ {
        sum += coefficients[u][v] * dctBasis[u][x]
      }
      columns[v][x] = sum
    }
  }

  var block [blockSide][blockSide]float64

  for y := 0; y < blockSide; y++ {
    for x := 0; x < blockSide; x++ {
      var sum float64
      for v := 0; v < blockSide; v++ {
        sum += columns[v][x] * dctBasis[v][y]
      }
      block[y][x] = sum / 4
    }
  }

  return block
}
```

Листинг А.2 – Встраивание, извлечение и статистика

```
package stego

import (
  "bytes"
  "encoding/binary"
  "errors"
  "fmt"
  "image"
  "image/color"
  "image/draw"
  _ "image/jpeg"
  "image/png"
  "math"
  "os"
  "path/filepath"
  "runtime"
  "strings"
  "sync"
```

```
)

// quantizationStep — шаг квантования QIM. Больше шаг — устойчивее бит к
// повторному сжатию и шуму, но заметнее искажение блока. При q = 16
// изменение среднего коэффициента не превышает 8, что на глаз незаметно.
const quantizationStep = 16.0

const (
  formatVersion = 1
  headerSize    = len(magic) + 1 + 4
)

// magic отделяет наше сообщение от произвольных битов, которые можно
// прочитать из коэффициентов обычной фотографии.
var magic = [7]byte{'M', 'Z', 'I', 'S', 'T', 'E', 'G'}

var (
  ErrNilImage         = errors.New("stego: nil image")
  ErrPayloadTooLarge  = errors.New("stego: payload does not fit into image")
  ErrNoPayload        = errors.New("stego: no MZISTEG payload found")
  ErrBadVersion       = errors.New("stego: unsupported payload version")
   ErrCorruptPayload    = errors.New("stego: payload length is inconsistent
with capacity")
  ErrSelfCheck         = errors.New("stego: embedded payload does not survive
PNG round trip")
  ErrMessageTooLong   = errors.New("stego: message is too long")
  ErrUnsupportedInput = errors.New("stego: unsupported image format")
)

// coefficient задаёт позицию среднечастотного коэффициента DCT. Низкие
// частоты слишком заметны для глаза, высокие — неустойчивы к сжатию,
// поэтому выбраны два симметричных среднечастотных коэффициента.
type coefficient struct {
  u, v int
}

var payloadCoefficients = [2]coefficient{
  {u: 3, v: 4},
  {u: 4, v: 3},
}

// Stats описывает результат встраивания и заполняется так, чтобы интерфейс
// мог показать цену сокрытия: ёмкость, искажение и число изменённых пикселей.
type Stats struct {
  Width         int
  Height        int
  CapacityBits  int
  UsedBits      int
  Blocks        int
  UsedBlocks    int
  ChangedBlocks int
  ChangedPixels int
  MaxDelta      float64
  MeanDelta     float64
  PSNR          float64
}

// Capacity возвращает ёмкость изображения в битах: по два бита на каждый
// полный блок 8×8. Неполные блоки по краям не используются.
func Capacity(img image.Image) int {
  if img == nil {
    return 0
  }
```

```
  bounds := img.Bounds()
  return 2 * (bounds.Dx() / blockSide) * (bounds.Dy() / blockSide)
}

// DecodeFile читает изображение из файла. Поддерживаются JPEG и PNG:
// JPEG — исходный контейнер, PNG — контейнер, в который мы записываем
// результат без повторного квантования.
func DecodeFile(path string) (image.Image, error) {
  file, err := os.Open(path)
  if err != nil {
    return nil, fmt.Errorf("open image: %w", err)
  }
  defer file.Close()

  img, format, err := image.Decode(file)
  if err != nil {
    return nil, fmt.Errorf("decode image: %w", err)
  }
  if format != "jpeg" && format != "png" {
    return nil, fmt.Errorf("%w: %s", ErrUnsupportedInput, format)
  }
  return img, nil
}

// WritePNG сохраняет контейнер без потерь. Повторное сохранение в JPEG
// недопустимо: квантование JPEG изменит коэффициенты и разрушит биты.
func WritePNG(path string, img image.Image) error {
  file, err := os.Create(path)
  if err != nil {
    return fmt.Errorf("create container: %w", err)
  }

  if err := png.Encode(file, img); err != nil {
    file.Close()
    return fmt.Errorf("encode container: %w", err)
  }
  if err := file.Close(); err != nil {
    return fmt.Errorf("close container: %w", err)
  }
  return nil
}

// OutputPath строит имя контейнера рядом с исходным файлом:
// photo.jpg → stego-photo.png.
func OutputPath(sourcePath string) string {
  dir := filepath.Dir(sourcePath)
  name := filepath.Base(sourcePath)
  ext := filepath.Ext(name)
  return filepath.Join(dir, "stego-"+strings.TrimSuffix(name, ext)+".png")
}

// Embed скрывает сообщение в среднечастотных коэффициентах DCT яркости.
// Возвращается изображение RGBA, которое нужно сохранять только как PNG.
func Embed(img image.Image, message []byte) (*image.RGBA, Stats, error) {
  if img == nil {
    return nil, Stats{}, ErrNilImage
  }
  if uint64(len(message)) > math.MaxUint32 {
    return nil, Stats{}, ErrMessageTooLong
  }

  bounds := img.Bounds()
  width, height := bounds.Dx(), bounds.Dy()
```

```
  columns, rows := width/blockSide, height/blockSide
  capacity := 2 * columns * rows

  payload := frame(message)
  bits := bytesToBits(payload)

  stats := Stats{
    Width:        width,
    Height:       height,
    CapacityBits: capacity,
    UsedBits:     len(bits),
    Blocks:       columns * rows,
  }
  if len(bits) > capacity {
    return nil, stats, fmt.Errorf(
      "%w: need %d bits, capacity is %d bits",
      ErrPayloadTooLarge, len(bits), capacity,
    )
  }

  stego := image.NewRGBA(image.Rect(0, 0, width, height))
  source := toRGBA(img)
  copy(stego.Pix, source.Pix)

  usedBlocks := embedBlocks(source, stego, columns, rows, bits)

  stats.UsedBlocks = usedBlocks
  stats.ChangedBlocks = usedBlocks
          stats.ChangedPixels,     stats.MaxDelta,     stats.MeanDelta     =
measureDifference(source, stego)
  stats.PSNR = psnr(source, stego)

  if err := verifyRoundTrip(stego, message); err != nil {
    return nil, stats, err
  }

  return stego, stats, nil
}

// embedBlocks раскладывает биты по блокам слева направо и сверху вниз. Блоки
// друг от друга не зависят, поэтому строки блоков делятся на полосы, и каждая
// полоса обрабатывается своей горутиной: номер блока задаёт его биты, так что
// синхронизация не нужна. На 4K это сокращает время с секунд до десятых.
func embedBlocks(source, stego *image.RGBA, columns, rows int, bits []byte)
int {
  workers := blockWorkers(rows)
  used := make([]int, workers)

  var wg sync.WaitGroup
  for worker := 0; worker < workers; worker++ {
    start := rows * worker / workers
    end := rows * (worker + 1) / workers

    wg.Add(1)
    go func(slot, start, end int) {
      defer wg.Done()
      used[slot] = embedBand(source, stego, columns, start, end, bits)
    }(worker, start, end)
  }
  wg.Wait()

  total := 0
  for _, count := range used {
```

```
    total += count
  }
  return total
}

func embedBand(source, stego *image.RGBA, columns, startRow, endRow int, bits
[]byte) int {
  used := 0
  for blockY := startRow; blockY < endRow; blockY++ {
    for blockX := 0; blockX < columns; blockX++ {
      bitIndex := 2 * (blockY*columns + blockX)
      if bitIndex >= len(bits) {
        return used
      }

      x0, y0 := blockX*blockSide, blockY*blockSide
      original := readLumaBlock(source, x0, y0)
      coefficients := forwardDCT(original)

      for i, position := range payloadCoefficients {
        if bitIndex+i >= len(bits) {
          break
        }
        coefficients[position.u][position.v] = embedBit(
          coefficients[position.u][position.v], bits[bitIndex+i],
        )
      }

      rebuilt := inverseDCT(coefficients)
      writeBlockDelta(stego, source, x0, y0, original, rebuilt)
      used++
    }
  }
  return used
}

// Extract читает сообщение из коэффициентов изображения.
func Extract(img image.Image) ([]byte, error) {
  if img == nil {
    return nil, ErrNilImage
  }

  source := toRGBA(img)
    columns,  rows  :=  source.Bounds().Dx()/blockSide,  source.Bounds().Dy()/
blockSide
  capacity := 2 * columns * rows
  if capacity < headerSize*8 {
    return nil, ErrNoPayload
  }

  bits := make([]byte, capacity)
  extractBits(source, columns, rows, bits)

  if !bytes.Equal(bitsToBytes(bits[:len(magic)*8]), magic[:]) {
    return nil, ErrNoPayload
  }

  version := bitsToBytes(bits[len(magic)*8 : len(magic)*8+8])[0]
  if version != formatVersion {
      return nil, fmt.Errorf("%w: got %d, want %d", ErrBadVersion, version,
formatVersion)
  }
```

```
  lengthStart := (len(magic) + 1) * 8
  lengthBits := bits[lengthStart : lengthStart+32]
  length := int(binary.BigEndian.Uint32(bitsToBytes(lengthBits)))

  maxPayload := capacity/8 - headerSize
  if length > maxPayload {
    return nil, fmt.Errorf(
      "%w: declared %d bytes, maximum is %d",
      ErrCorruptPayload, length, maxPayload,
    )
  }

  end := (headerSize + length) * 8
  return bitsToBytes(bits[headerSize*8 : end]), nil
}

// embedBit ставит коэффициент в центр квантованной ячейки, номер которой
// имеет нужную чётность. Решающая граница находится на расстоянии q/2 от
// центра, поэтому ошибка округления при обратном DCT и сохранении в PNG
// не может перевернуть бит.
func embedBit(value float64, bit byte) float64 {
  index := int(math.Round(value / quantizationStep))
  if parity(index) != bit {
    lower := float64(index-1) * quantizationStep
    upper := float64(index+1) * quantizationStep
    if math.Abs(value-lower) < math.Abs(value-upper) {
      index--
    } else {
      index++
    }
  }
  return float64(index) * quantizationStep
}

// readBit повторяет квантование и возвращает чётность номера ячейки.
func readBit(value float64) byte {
  return parity(int(math.Round(value / quantizationStep)))
}

func parity(index int) byte {
  return byte(((index % 2) + 2) % 2)
}

func verifyRoundTrip(stego *image.RGBA, message []byte) error {
  var encoded bytes.Buffer
  if err := png.Encode(&encoded, stego); err != nil {
    return fmt.Errorf("%w: encode PNG: %v", ErrSelfCheck, err)
  }

  decoded, err := png.Decode(&encoded)
  if err != nil {
    return fmt.Errorf("%w: decode PNG: %v", ErrSelfCheck, err)
  }

  extracted, err := Extract(decoded)
  if err != nil {
    if errors.Is(err, ErrNoPayload) {
      return fmt.Errorf(
         "%w: areas of pure black or white clip the brightness change; try a
picture with more headroom",
        ErrSelfCheck,
      )
    }
```

```
    return fmt.Errorf("%w: extraction after saving failed: %w", ErrSelfCheck,
err)
  }
  if !bytes.Equal(extracted, message) {
    return fmt.Errorf("%w: payload changed after PNG round trip", ErrSelfCheck)
  }
  return nil
}

func frame(message []byte) []byte {
  payload := make([]byte, headerSize+len(message))
  copy(payload, magic[:])
  payload[len(magic)] = formatVersion
  binary.BigEndian.PutUint32(payload[len(magic)+1:], uint32(len(message)))
  copy(payload[headerSize:], message)
  return payload
}

func bytesToBits(data []byte) []byte {
  bits := make([]byte, 0, len(data)*8)
  for _, value := range data {
    for shift := 7; shift >= 0; shift-- {
      bits = append(bits, (value>>uint(shift))&1)
    }
  }
  return bits
}

func bitsToBytes(bits []byte) []byte {
  out := make([]byte, len(bits)/8)
  for i := range out {
    for bit := 0; bit < 8; bit++ {
      out[i] = out[i]<<1 | bits[i*8+bit]
    }
  }
  return out
}

// quote: helpers below work on *image.RGBA.

// toRGBA приводит изображение к RGBA: дальше пиксели читаются и пишутся по
// слайсу Pix, без интерфейсных вызовов At/Set на каждый пиксель.
func toRGBA(img image.Image) *image.RGBA {
  if rgba, ok := img.(*image.RGBA); ok {
    return rgba
  }
  bounds := img.Bounds()
  out := image.NewRGBA(image.Rect(0, 0, bounds.Dx(), bounds.Dy()))
  draw.Draw(out, out.Bounds(), img, bounds.Min, draw.Src)
  return out
}

// blockWorkers ограничивает число горутин числом полос и ядер.
func blockWorkers(rows int) int {
  workers := runtime.NumCPU()
  if workers > rows {
    workers = rows
  }
  if workers < 1 {
    workers = 1
  }
  return workers
}
```

```
// extractBits читает биты из коэффициентов блоков — та же разбивка на полосы,
// что и при встраивании, но только чтение.
func extractBits(source *image.RGBA, columns, rows int, bits []byte) {
  workers := blockWorkers(rows)

  var wg sync.WaitGroup
  for worker := 0; worker < workers; worker++ {
    start := rows * worker / workers
    end := rows * (worker + 1) / workers

    wg.Add(1)
    go func(start, end int) {
      defer wg.Done()
      for blockY := start; blockY < end; blockY++ {
        for blockX := 0; blockX < columns; blockX++ {
           coefficients := forwardDCT(readLumaBlock(source, blockX*blockSide,
blockY*blockSide))
          index := 2 * (blockY*columns + blockX)
          for i, position := range payloadCoefficients {
            bits[index+i] = readBit(coefficients[position.u][position.v])
          }
        }
      }
    }(start, end)
  }
  wg.Wait()
}

// readLumaBlock читает блок 8×8 и переводит пиксели в яркость.
func readLumaBlock(img *image.RGBA, x0, y0 int) [blockSide][blockSide]float64 {
  var block [blockSide][blockSide]float64
  for y := 0; y < blockSide; y++ {
    offset := img.PixOffset(x0, y0+y)
    for x := 0; x < blockSide; x++ {
      p := offset + 4*x
      block[y][x] = lumaRGB(img.Pix[p], img.Pix[p+1], img.Pix[p+2])
    }
  }
  return block
}

// writeBlockDelta добавляет к R, G и B одну и ту же разность яркостей.
// Прибавка ко всем каналам сохраняет оттенок, а яркость результата
// совпадает с восстановленной яркостью блока с точностью до округления.
func writeBlockDelta(stego, source *image.RGBA, x0, y0 int, original, rebuilt
[blockSide][blockSide]float64) {
  for y := 0; y < blockSide; y++ {
    offset := source.PixOffset(x0, y0+y)
    for x := 0; x < blockSide; x++ {
      p := offset + 4*x
      delta := rebuilt[y][x] - original[y][x]
      stego.Pix[p] = clampChannel(float64(source.Pix[p]) + delta)
      stego.Pix[p+1] = clampChannel(float64(source.Pix[p+1]) + delta)
      stego.Pix[p+2] = clampChannel(float64(source.Pix[p+2]) + delta)
      stego.Pix[p+3] = source.Pix[p+3]
    }
  }
}

func lumaRGB(r, g, b uint8) float64 {
  return 0.299*float64(r) + 0.587*float64(g) + 0.114*float64(b)
}
```

```
// Luma переводит пиксель в яркость по формуле JPEG. Одинаковая формула
// используется и при встраивании, и при извлечении.
func Luma(pixel color.Color) float64 {
  value := color.NRGBAModel.Convert(pixel).(color.NRGBA)
  return lumaRGB(value.R, value.G, value.B)
}

func lumaRGBFrom(pixel color.Color) float64 {
  value := color.NRGBAModel.Convert(pixel).(color.NRGBA)
  return lumaRGB(value.R, value.G, value.B)
}

func clampChannel(value float64) uint8 {
  rounded := math.Round(value)
  if rounded < 0 {
    return 0
  }
  if rounded > 255 {
    return 255
  }
  return uint8(rounded)
}
```